

Deep Learning

Complete Notes + PYQ Solutions

IIT Madras BS Data Science Program

McCulloch-Pitts → Perceptrons → Backprop → CNNs
→ RNNs → LSTMs → Attention → Transformers

All PYQ Papers Solved

End Term Jan 2026 · Quiz 1 & 2 Jan 2026 · Term 2 & 3 2025

Based on course notes by Rishabh Indoria (21F3001823).

Expanded, structured, and solved by:

ATHARV KHARE

with AI assistance · 2026

How to Use This Book

This guide is structured for exam success. Each chapter follows a consistent pattern:

1. **Motivation** — why this concept exists
2. **Intuition** — the core idea in plain language
3. **Formal definition** — precise mathematical statement
4. **Deep explanation** — step-by-step derivations
5. **Algorithms** — pseudocode you can trace by hand
6. **Exam tips, memory tricks, common mistakes**
7. **Quick revision bullets** at chapter end

Exam Tip

Box colour guide:

- **Green border** — Exam Tip (directly tested patterns)
- **Amber border** — Intuition
- **Red border** — Common Mistake to avoid
- **Purple border** — Memory Trick
- **Dark green border** — Quick Revision

Contents

How to Use This Book	ii
1 History of Deep Learning	1
1.1 Biological Inspiration	1
1.2 From Spring to Winter of AI	1
1.3 The Deep Revival	1
1.4 Milestones Timeline	2
1.5 Key Eras	2
1.5.1 Sequences & RNNs	2
1.5.2 The Rise of Transformers	2
1.5.3 GANs & Diffusion	2
1.5.4 Reinforcement Learning	2
1.5.5 AI in Science	2
2 McCulloch-Pitts Neuron & Perceptrons	3
2.1 McCulloch-Pitts (MP) Neuron	3
2.1.1 Implementing Boolean Functions	3
2.2 The Perceptron (Rosenblatt, 1958)	3
2.3 Perceptron Learning Algorithm	4
2.3.1 Why does the update work?	4
2.3.2 Convergence Proof Sketch	4
2.4 Network of Perceptrons	5
3 Sigmoid Neurons & Supervised Learning Setup	6
3.1 Why Sigmoid?	6
3.2 Supervised Machine Learning Setup	6
3.3 Gradient Descent for Sigmoid Neuron	6
4 Feed-Forward Neural Networks & Backpropagation	8
4.1 Structure of a FFNN	8
4.1.1 Output and Loss Functions	8
4.2 Backpropagation	8
4.2.1 Step 1 — Gradient w.r.t. Output Layer (Softmax + Cross-Entropy)	8
4.2.2 Step 2 — Gradient w.r.t. Hidden Units (Chain Rule)	9
4.2.3 Step 3 — Gradient w.r.t. Parameters	9
4.2.4 Full Pseudocode	9
5 Gradient Descent Variants	11
5.1 Vanilla (Batch) Gradient Descent	11
5.2 Stochastic & Mini-Batch GD	11
5.3 Momentum-Based GD	11
5.4 Nesterov Accelerated Gradient (NAG)	12
5.5 Learning Rate Scheduling	12

6	Adaptive Learning Rates	14
6.1	AdaGrad	14
6.2	RMSprop	14
6.3	AdaDelta	14
6.4	Adam (Adaptive Moments)	15
6.4.1	AdaMax	15
6.4.2	NAdam	15
7	Regularisation	17
7.1	Bias and Variance	17
7.2	L2 Regularisation (Weight Decay)	17
7.3	Data Augmentation	17
7.4	Parameter Sharing	18
7.5	Dropout	18
7.6	Early Stopping	18
7.7	Ensemble Methods & Bagging	18
8	Activation Functions & Deep Revival	20
8.1	Problems with Sigmoid & Tanh	20
8.2	ReLU Family	20
8.3	Unsupervised Pre-Training (Hinton, 2006)	21
8.4	Weight Initialisation	21
9	Convolutional Neural Networks	22
9.1	Motivation	22
9.2	Convolution Operation	22
9.3	Output Dimensions	22
9.4	Pooling	22
9.5	Backpropagation in CNNs	23
9.6	Famous Architectures	23
10	Word Representations	24
10.1	The Problem with One-Hot Vectors	24
10.2	Co-occurrence Matrix	24
10.2.1	SVD for Dense Representations	24
10.3	Prediction-Based Models (Neural Word Embeddings)	25
10.3.1	Bag-of-Words (CBOW)	25
10.3.2	Skip-Gram	25
10.3.3	Negative Sampling	25
10.3.4	Hierarchical Softmax	25
10.4	GloVe (Global Vectors)	25
11	Recurrent Neural Networks	27
11.1	Why RNNs?	27
11.2	RNN Architecture	27
11.3	Backpropagation Through Time (BPTT)	27
11.4	RNN Modes	28

12 LSTM & GRU	29
12.1 The Problem LSTM Solves	29
12.2 Long Short-Term Memory (LSTM)	29
12.3 Gated Recurrent Unit (GRU)	30
12.4 Vanishing Gradient in LSTM — Forward vs. Backward	30
13 Encoder-Decoder Models & Attention	31
13.1 Encoder-Decoder Architecture	31
13.2 Bottleneck Problem	31
13.3 Attention Mechanism	31
13.4 Attention for Images	32
13.5 Hierarchical Attention	32
14 Transformer Architecture	33
14.1 Motivation	33
14.2 Self-Attention	33
14.3 Multi-Headed Attention	33
14.4 Positional Encoding	33
14.5 Feed-Forward Sub-layer	34
14.6 Encoder Architecture	34
14.7 Decoder Architecture	34
Quick Reference Cheat Sheet	36
15 PYQ Solutions	37
15.1 End Term Jan 2026 — With Solutions	37
15.2 Term 3 2025 — Key Questions	40
15.3 Quiz Papers — Key Questions	40
Index of Key Formulas	42

History of Deep Learning

1.1 Biological Inspiration

Deep learning traces its roots to attempts to model the brain computationally.

- **Reticular Theory** (Golgi): nervous system is *one continuous network*.
- **Neuron Doctrine** (Ramón y Cajal, staining technique): nervous system is made of *discrete individual cells*.
- The neuron doctrine won — and inspired the artificial neuron.

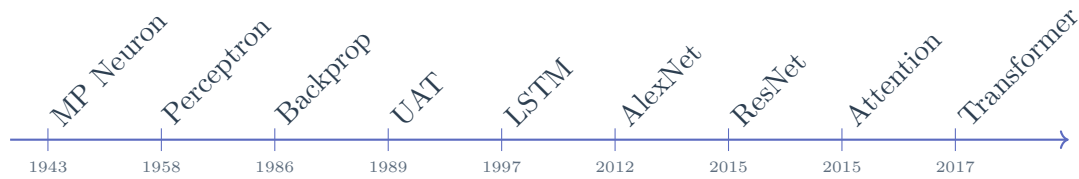
1.2 From Spring to Winter of AI

Milestone	Significance
McCulloch-Pitts (1943)	Simplified computational model of a neuron
Perceptron (Rosenblatt, 1958)	Learning weights; “will walk, talk, see...”
<i>Perceptrons</i> book (1969)	Minsky & Papert showed XOR cannot be solved — AI Winter
Backpropagation (1986)	Rumelhart et al. — efficient gradient computation
Universal Approx. Theorem	Single hidden layer can approximate any continuous function

1.3 The Deep Revival

- **Unsupervised Pre-Training** (Hinton, 2006): initialise deep nets greedily layer-by-layer, then fine-tune.
- **Hubel & Wiesel**: receptive fields → motivation for CNNs.
- **Better optimisers**: AdaGrad, RMSprop, Adam, Nadam.
- **Better activations**: tanh → ReLU → Leaky ReLU → GELU.

1.4 Milestones Timeline



1.5 Key Eras

1.5.1 Sequences & RNNs

Hopfield networks (content-addressable memory) → Jordan/Elman networks → LSTM (Hochreiter & Schmidhuber, 1997) solved the vanishing gradient problem for long sequences.

1.5.2 The Rise of Transformers

Rule-based MT → Statistical MT (IBM models) → Seq2Seq with attention → Transformer (2017) → BERT, GPT.

1.5.3 GANs & Diffusion

From discriminative → generative. VAE, GAN (unstable, mode collapse) → Diffusion models (non-equilibrium thermodynamics, stable, diverse).

1.5.4 Reinforcement Learning

DQN (Atari), AlphaGo/AlphaZero, MuZero (no rules given), Player of Games (perfect + imperfect information).

1.5.5 AI in Science

AlphaFold (protein structure), astronomy (galaxy evolution prediction), Analog AI (programmable resistors).

Quick Revision

- Neuron Doctrine $\neq$ Reticular Theory — neurons are *discrete*.
- Minsky & Papert killed Perceptron hype (XOR problem); backprop revived it.
- Three DL revivals: backprop (1986), unsupervised pre-training (2006), AlexNet/GPUs (2012).
- Transformer (2017) parallelises what RNNs did sequentially.

McCulloch-Pitts Neuron & Perceptrons

2.1 McCulloch-Pitts (MP) Neuron

Definition

An MP neuron aggregates binary inputs and fires if the aggregation meets a threshold:

$$g(\mathbf{x}) = \sum_{i=1}^n x_i, \quad y = f(g(\mathbf{x})) = \begin{cases} 1 & g(\mathbf{x}) \geq \theta \\ 0 & \text{otherwise} \end{cases}$$

Inputs are either **excitatory** (count toward sum) or **inhibitory** (if any inhibitory input = 1, then $y = 0$).

2.1.1 Implementing Boolean Functions

An MP neuron can implement any *linearly separable* boolean function.

Function	Inputs	θ	Inhibitory?
AND(x_1, x_2)	both excitatory	2	No
OR(x_1, x_2)	both excitatory	1	No
NOT(x_1)	x_1 inhibitory	0	Yes (x_1)

Common Mistake

XOR is **not** linearly separable, so a single MP neuron *cannot* implement it. For n binary inputs there are 2^{2^n} boolean functions; only 14 of 16 two-input functions are linearly separable.

2.2 The Perceptron (Rosenblatt, 1958)

Definition

The perceptron extends the MP neuron by introducing **real-valued learnable**

weights:

$$y = \begin{cases} 1 & \sum_{i=0}^n w_i x_i \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

where $w_0 = -\theta$ is the **bias** (prior/prejudice) and $x_0 = 1$.

Intuition

The weight vector $\mathbf{w}$ is **perpendicular** to the decision boundary $\mathbf{w}^T \mathbf{x} = 0$. Any point $\mathbf{x}$ on the boundary satisfies $\cos \alpha = \frac{\mathbf{w}^T \mathbf{x}}{\|\mathbf{w}\| \|\mathbf{x}\|} = 0$. Points with $\mathbf{w}^T \mathbf{x} > 0$ lie on the same side as $\mathbf{w}$; points with $\mathbf{w}^T \mathbf{x} < 0$ lie on the opposite side.

2.3 Perceptron Learning Algorithm

Algorithm 1: Perceptron Learning

Input: Positive set P (label 1), Negative set N (label 0)

Output: Weight vector $\mathbf{w}$

```

1 Initialise  $\mathbf{w}$  randomly;
2 while not converged do
3   Pick random  $\mathbf{x} \in P \cup N$ ;
4   if  $\mathbf{x} \in P$  and  $\mathbf{w}^T \mathbf{x} < 0$  then
5      $\mathbf{w} \leftarrow \mathbf{w} + \mathbf{x}$ ;
6   end
7   if  $\mathbf{x} \in N$  and  $\mathbf{w}^T \mathbf{x} \geq 0$  then
8      $\mathbf{w} \leftarrow \mathbf{w} - \mathbf{x}$ ;
9   end
10 end
```

2.3.1 Why does the update work?

For $\mathbf{x} \in P$ misclassified ($\mathbf{w}^T \mathbf{x} < 0$, i.e. $\alpha > 90^\circ$), after update:

$$\mathbf{w}_{\text{new}}^T \mathbf{x} = (\mathbf{w} + \mathbf{x})^T \mathbf{x} = \mathbf{w}^T \mathbf{x} + \|\mathbf{x}\|^2 > \mathbf{w}^T \mathbf{x}$$

The dot product increases, decreasing α toward $< 90^\circ$. Similarly for N .

2.3.2 Convergence Proof Sketch

Let $\mathbf{w}^*$ be the normalised solution. After k corrections:

$$\cos \beta = \frac{\mathbf{w}^* \cdot \mathbf{w}_k}{\|\mathbf{w}_k\|} \geq \frac{\mathbf{w}^* \cdot \mathbf{w}_0 + k\delta}{\sqrt{\|\mathbf{w}_0\|^2 + k}} \propto \sqrt{k}$$

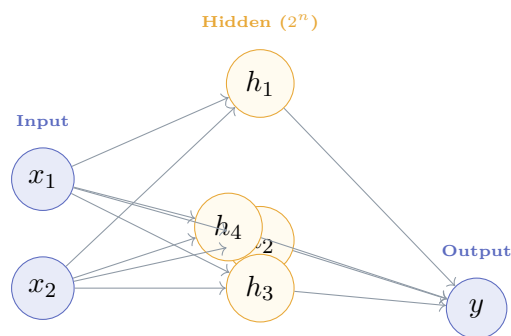
Since $\cos \beta \leq 1$, k is bounded — the algorithm **always converges** if P, N are linearly separable.

Exam Tip

Perceptron Convergence Theorem: If the data is linearly separable, the algorithm converges in a *finite* number of steps. If not, it runs forever — this is how you detect non-separability.

2.4 Network of Perceptrons

A single hidden layer with 2^n perceptrons can implement **any** boolean function of n inputs.

**Quick Revision**

- MP neuron: binary inputs, fixed θ , no learning.
- Perceptron: real inputs, learnable weights, bias $w_0 = -\theta$.
- Both can only separate *linearly separable* data.
- Two-layer network of perceptrons: universal for boolean functions.
- 2^{2^n} boolean functions of n inputs; requires 2^n hidden neurons.

Sigmoid Neurons & Supervised Learning Setup

3.1 Why Sigmoid?

The perceptron's step function is **non-differentiable** — we can't do calculus on it. We need a *smooth* decision boundary.

Definition

Sigmoid (Logistic) Neuron:

$$y = \sigma(\mathbf{w}^T \mathbf{x}) = \frac{1}{1 + e^{-(w_0 + \sum_{i=1}^n w_i x_i)}}$$

Output is in $(0, 1)$, interpretable as a probability. Smooth $\Rightarrow$ differentiable everywhere.

Key property: $\frac{d\sigma}{dx} = \sigma(x)(1 - \sigma(x))$.

3.2 Supervised Machine Learning Setup

Component	Description
Data	$\{(\mathbf{x}_i, y_i)\}_{i=1}^N$, where $\mathbf{x}_i \in \mathbb{R}^m$
Model	$\hat{y} = \hat{f}(\mathbf{x}; \theta)$, e.g. $\hat{y} = \sigma(\mathbf{w}^T \mathbf{x})$
Loss	$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$ (MSE)
Goal	Find $\theta^* = \arg \min_{\theta} \mathcal{L}(\theta)$

3.3 Gradient Descent for Sigmoid Neuron

Using Taylor series, the direction that decreases $\mathcal{L}$ most is $-\nabla_{\theta} \mathcal{L}$:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta_t} \mathcal{L}$$

For a single sigmoid neuron $f(x) = \sigma(wx + b)$:

$$\frac{\partial \mathcal{L}}{\partial w} = (f(x) - y) \cdot f(x) \cdot (1 - f(x)) \cdot x$$

$$\frac{\partial \mathcal{L}}{\partial b} = (f(x) - y) \cdot f(x) \cdot (1 - f(x))$$

Algorithm 2: Gradient Descent (Sigmoid Neuron)

Input: Training data $\{(\mathbf{x}_i, y_i)\}$, learning rate η , max_iter

```

1 Initialise  $w, b$  randomly;
2 for  $t = 1$  to max_iter do
3    $w \leftarrow w - \eta \cdot \nabla_w \mathcal{L}$ ;
4    $b \leftarrow b - \eta \cdot \nabla_b \mathcal{L}$ ;
5 end
```

Intuition

Gradient descent is like a ball rolling downhill on the loss surface. The gradient $\nabla \mathcal{L}$ points uphill; we step in the *opposite* direction by η (learning rate).

Exam Tip

Universal Approximation Theorem: A single-hidden-layer network of sigmoid neurons can approximate *any* continuous function to arbitrary precision — but may need exponentially many hidden neurons.

Quick Revision

- Sigmoid smooths the perceptron step function.
- $\sigma'(x) = \sigma(x)(1 - \sigma(x))$ — memorise this!
- MSE loss for regression; cross-entropy for classification.
- Learning rate η too large $\Rightarrow$ diverge; too small $\Rightarrow$ too slow.

Feed-Forward Neural Networks & Backpropagation

4.1 Structure of a FFNN

Definition

A **Feed-Forward Neural Network (FFNN)** with L layers:

$$\begin{aligned}\mathbf{a}_k &= \mathbf{b}_k + W_k \mathbf{h}_{k-1} && \text{(pre-activation)} \\ \mathbf{h}_k &= g(\mathbf{a}_k) && \text{(activation, element-wise)} \\ \hat{\mathbf{y}} &= \mathbf{h}_L = O(\mathbf{a}_L) && \text{(output layer)}\end{aligned}$$

where $\mathbf{h}_0 = \mathbf{x}$, $W_k \in \mathbb{R}^{n_k \times n_{k-1}}$, $\mathbf{b}_k \in \mathbb{R}^{n_k}$.

Parameters: $\theta = \{W_1, \dots, W_L, \mathbf{b}_1, \dots, \mathbf{b}_L\}$.

4.1.1 Output and Loss Functions

Task	Output Activation O	Loss
Regression	Linear: $O(\mathbf{a}_L) = W_O \mathbf{a}_L + \mathbf{b}_O$	MSE: $\frac{1}{N} \sum \ \hat{\mathbf{y}} - \mathbf{y}\ ^2$
Multi-class	Softmax: $\hat{y}_j = \frac{e^{a_{L,j}}}{\sum_i e^{a_{L,i}}}$	Cross-entropy: $-\frac{1}{N} \sum \mathbf{y} \log \hat{\mathbf{y}}$
Binary	Sigmoid	Binary cross-entropy

4.2 Backpropagation

Intuition

Backprop is **chain rule applied systematically**. Think of it as “talking” to each layer in reverse order: output layer $\rightarrow$ last hidden $\rightarrow \dots \rightarrow$ first hidden $\rightarrow$ parameters.

4.2.1 Step 1 — Gradient w.r.t. Output Layer (Softmax + Cross-Entropy)

Using $\mathcal{L} = -\log \hat{y}_\ell$ where ℓ is the true class:

$$\nabla_{\mathbf{a}_L} \mathcal{L} = \hat{\mathbf{y}} - \mathbf{e}_\ell \quad (\text{predicted prob. minus one-hot true})$$

4.2.2 Step 2 — Gradient w.r.t. Hidden Units (Chain Rule)

$$\frac{\partial \mathcal{L}}{\partial h_{k,j}} = \sum_m \frac{\partial \mathcal{L}}{\partial a_{k+1,m}} W_{k+1,m,j} \quad \Rightarrow \quad \nabla_{\mathbf{h}_k} \mathcal{L} = W_{k+1}^T \nabla_{\mathbf{a}_{k+1}} \mathcal{L}$$

$$\nabla_{\mathbf{a}_k} \mathcal{L} = \nabla_{\mathbf{h}_k} \mathcal{L} \odot g'(\mathbf{a}_k)$$

4.2.3 Step 3 — Gradient w.r.t. Parameters

$$\nabla_{W_k} \mathcal{L} = \nabla_{\mathbf{a}_k} \mathcal{L} \cdot \mathbf{h}_{k-1}^T \quad \nabla_{\mathbf{b}_k} \mathcal{L} = \nabla_{\mathbf{a}_k} \mathcal{L}$$

4.2.4 Full Pseudocode

Algorithm 3: Forward & Backward Propagation

```

1 Function ForwardPass(x):
2   h0 ← x;
3   for k = 1 to L − 1 do
4     ak ← bk + Wkhk−1;
5     hk ← g(ak);
6   end
7   aL ← bL + WLhL−1; ŷ ← softmax(aL);
8 Function BackwardPass(ŷ, y):
9   ∇aL  $\mathcal{L}$  ← ŷ − ey;
10  for k = L to 1 do
11    ∇Wk  $\mathcal{L}$  ← ∇ak  $\mathcal{L}$  · hk−1T;
12    ∇bk  $\mathcal{L}$  ← ∇ak  $\mathcal{L}$ ;
13    ∇hk−1  $\mathcal{L}$  ← WkT ∇ak  $\mathcal{L}$ ;
14    ∇ak−1  $\mathcal{L}$  ← ∇hk−1  $\mathcal{L}$  ⊙ g'(ak−1);
15  end

```

Exam Tip

Gradient of softmax + cross-entropy: $\nabla_{\mathbf{a}_L} = \hat{\mathbf{y}} - \mathbf{e}_y$. This is the cleanest gradient in deep learning — memorise it.

Dimensions check: $\nabla_{W_k} = \underbrace{\nabla_{\mathbf{a}_k}}_{n_k \times 1} \cdot \underbrace{\mathbf{h}_{k-1}^T}_{1 \times n_{k-1}} = n_k \times n_{k-1}$

Common Mistake

g' for common activations:

- Sigmoid σ : $g'(a) = \sigma(a)(1 - \sigma(a))$
- Tanh: $g'(a) = 1 - \tanh^2(a)$
- ReLU: $g'(a) = \mathbf{1}[a > 0]$ (0 for $a \leq 0$ — dead neuron!)

Quick Revision

- FFNN: pre-activation $\mathbf{a}_k$, activation $\mathbf{h}_k = g(\mathbf{a}_k)$.

- Backprop = forward pass + reverse chain rule.
- Output gradient (softmax+CE): $\hat{\mathbf{y}} - \mathbf{e}_y$ (simple!).
- Parameter update: $\nabla_{W_k} = \nabla_{\mathbf{a}_k} \mathbf{h}_{k-1}^T$.

Gradient Descent Variants

5.1 Vanilla (Batch) Gradient Descent

Compute gradient over the *entire* dataset, then update once:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}$$

Problem: $N = 1\text{M}$ points $\Rightarrow$ one update requires 1M gradient computations.

5.2 Stochastic & Mini-Batch GD

Algorithm	Steps / epoch	Properties
Batch GD	1	True gradient; slow per epoch
Stochastic GD (SGD)	N	Noisy; may not decrease every step
Mini-batch GD	N/B	Best of both; B typically 32–512

Definition

Terminology:

- **Epoch:** one full pass over training data.
- **Step / iteration:** one parameter update.
- **Batch size B :** number of samples per step.

5.3 Momentum-Based GD

Definition

Accumulate a **velocity** term to dampen oscillations and speed up convergence:

$$\mathbf{u}_t = \beta \mathbf{u}_{t-1} + \nabla_{\theta_t} \mathcal{L} \quad \theta_{t+1} = \theta_t - \eta \mathbf{u}_t$$

Expanding: $\mathbf{u}_t = \sum_{\tau=0}^t \beta^{t-\tau} \nabla_{\theta_{\tau}} \mathcal{L}$ (exponentially decaying history, $\beta < 1$).

Intuition

Like a heavy ball rolling downhill — gains momentum in consistent directions, resists random noise. On gentle slopes it keeps going; on steep slopes it doesn't shoot too far because of β .

Problem: Can overshoot and oscillate.

5.4 Nesterov Accelerated Gradient (NAG)

Definition

"Look before you leap": compute gradient at the *lookahead* position $\theta_t - \beta \mathbf{u}_{t-1}$:

$$\mathbf{u}_t = \beta \mathbf{u}_{t-1} + \nabla_{\theta_t - \beta \mathbf{u}_{t-1}} \mathcal{L} \quad \theta_{t+1} = \theta_t - \eta \mathbf{u}_t$$

NAG corrects for overshoot by computing a more informed gradient. Converges faster than momentum with fewer oscillations.

5.5 Learning Rate Scheduling

- **Step decay:** halve η every k epochs.
- **Exponential decay:** $\eta_t = \eta_0 e^{-kt}$.
- **1/t decay:** $\eta_t = \eta_0 / (1 + kt)$.
- **Cyclical LR:** increase and decrease η cyclically — escapes saddle points.
- **Cosine Annealing (Warm Restart):** $\eta_t = \eta_{\min} + \frac{\eta_{\max} - \eta_{\min}}{2} (1 + \cos \pi \frac{t \bmod T}{T})$

Exam Tip

Saddle point trap: decreasing-only LR can get stuck at saddle points. Cyclical LR (or cosine annealing) periodically raises η to escape.

Typical β values for momentum: 0.9 (safe default). For Sutskever's schedule: $\beta_{\max} \in \{0.999, 0.99, 0.9\}$.

Memory Trick

GD family tree:

Batch GD $\xrightarrow{+\text{velocity}}$ Momentum $\xrightarrow{+\text{lookahead}}$ NAG
 Too slow $\xrightarrow{+\text{mini-batch}}$ SGD variants
 Flat region? $\xrightarrow{+\text{cyclic LR}}$ Warm restarts

Quick Revision

- Batch: 1 step/epoch; SGD: N steps; Mini-batch: N/B steps.
- Momentum accumulates past gradients with decay β .
- NAG: compute gradient at future position (lookahead).

- Cyclical LR helps escape saddle points.

Adaptive Learning Rates

All methods in this chapter adapt the learning rate *per parameter* based on gradient history.

6.1 AdaGrad

Definition

Accumulate squared gradients; divide LR by their root:

$$\mathbf{v}_t = \mathbf{v}_{t-1} + (\nabla\theta_t)^2, \quad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\mathbf{v}_t} + \epsilon} \nabla\theta_t$$

Intuition: parameters updated frequently get small LR; infrequently updated parameters get large LR.

Problem: $\mathbf{v}_t$ grows monotonically $\Rightarrow$ LR $\rightarrow 0$ too fast.

6.2 RMSprop

Definition

Replace running *sum* with *decaying average*:

$$\mathbf{v}_t = \beta\mathbf{v}_{t-1} + (1 - \beta)(\nabla\theta_t)^2, \quad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\mathbf{v}_t} + \epsilon} \nabla\theta_t$$

Prevents the denominator from growing indefinitely. LR can increase or decrease. Sensitive to initial LR.

6.3 AdaDelta

Avoids setting η altogether by tracking both gradient history *and* update history:

$$\begin{aligned} \mathbf{v}_t &= \beta\mathbf{v}_{t-1} + (1 - \beta)(\nabla\theta_t)^2 \\ \Delta\theta_t &= -\frac{\sqrt{\mathbf{u}_{t-1} + \epsilon}}{\sqrt{\mathbf{v}_t} + \epsilon} \nabla\theta_t \\ \mathbf{u}_t &= \beta\mathbf{u}_{t-1} + (1 - \beta)(\Delta\theta_t)^2 \end{aligned}$$

One history lags the other — the numerator acts as an adaptive step size.

6.4 Adam (Adaptive Moments)

Definition

Combines momentum (first moment $\mathbf{m}$) + RMSprop (second moment $\mathbf{v}$) + bias correction:

$$\begin{aligned}\mathbf{m}_t &= \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \nabla \theta_t & \hat{\mathbf{m}}_t &= \frac{\mathbf{m}_t}{1 - \beta_1^t} \\ \mathbf{v}_t &= \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) (\nabla \theta_t)^2 & \hat{\mathbf{v}}_t &= \frac{\mathbf{v}_t}{1 - \beta_2^t} \\ \theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \hat{\mathbf{m}}_t\end{aligned}$$

Typical defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, $\eta = 10^{-3}$.

Intuition

Bias correction: at $t = 1$, $\mathbf{m}_1 = (1 - \beta_1) \nabla \theta$ is tiny (biased toward 0). Dividing by $(1 - \beta_1^t)$ corrects this. Think of it as the expected value of the gradient rather than a noisy one-point estimate.

6.4.1 AdaMax

Replace L^2 norm in $\mathbf{v}$ with L^∞ norm:

$$\mathbf{v}_t = \max(\beta_2 \mathbf{v}_{t-1}, |\nabla \theta_t|), \quad \theta_{t+1} = \theta_t - \frac{\eta}{\mathbf{v}_t + \epsilon} \hat{\mathbf{m}}_t$$

No bias correction for $\mathbf{v}$ (max is not susceptible to initial zero bias).

6.4.2 NAdam

Incorporate NAG into Adam by replacing $\hat{\mathbf{m}}_t$ with the lookahead gradient estimate:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{\mathbf{v}}_{t+1}} + \epsilon} \left(\hat{\mathbf{m}}_{t+1} + \frac{(1 - \beta_1) \nabla \theta_t}{1 - \beta_1^{t+1}} \right)$$

Optimiser	Key idea	Limitation
AdaGrad	Accumulate $(\nabla)^2$	LR decays to 0
RMSprop	Decay $(\nabla)^2$ history	Sensitive to η_0
AdaDelta	No η_0 , two histories	Extra hyperparameter β
Adam	Momentum + RMSprop + bias-corr	Default choice; may overshoot
AdaMax	L^∞ norm	Niche use
NAdam	Adam + NAG lookahead	Slightly complex

Exam Tip

PYQ pattern: You will be asked to compute Adam update steps numerically. Know the formula for $\hat{\mathbf{m}}_t$ and $\hat{\mathbf{v}}_t$ cold.

Typical learning rates to try (log scale): $10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}$.

Quick Revision

- AdaGrad: $\mathbf{v}_t = \mathbf{v}_{t-1} + g_t^2$ (grows forever).
- RMSprop: $\mathbf{v}_t = \beta \mathbf{v}_{t-1} + (1 - \beta) g_t^2$ (decaying avg).
- Adam = momentum ($\mathbf{m}$) + RMSprop ($\mathbf{v}$) + bias correction ($\hat{\mathbf{m}}, \hat{\mathbf{v}}$).
- Bias correction: divide by $(1 - \beta^t)$ at step t .

Regularisation

7.1 Bias and Variance

Definition

For model $\hat{f}(\mathbf{x})$ trained on a dataset:

$$\text{Bias}[\hat{f}(\mathbf{x})] = \mathbb{E}[\hat{f}(\mathbf{x})] - f(\mathbf{x})$$

$$\text{Variance}[\hat{f}(\mathbf{x})] = \mathbb{E}[(\hat{f}(\mathbf{x}) - \mathbb{E}[\hat{f}(\mathbf{x})])^2]$$

$$\mathbb{E}[(y - \hat{f}(\mathbf{x}))^2] = \text{Bias}^2 + \text{Variance} + \sigma_{\text{noise}}^2$$

	Bias	Variance	Overfit?
Simple model	High	Low	Underfits
Complex model	Low	High	Overfits

Deep nets are **over-parameterised** (billions of params, millions of training points)
 $\Rightarrow$ need regularisation.

7.2 L2 Regularisation (Weight Decay)

Definition

Add penalty on weight magnitude to the loss:

$$\bar{\mathcal{L}}(\mathbf{w}) = \mathcal{L}(\mathbf{w}) + \frac{\alpha}{2} \|\mathbf{w}\|^2 \quad \Rightarrow \quad \nabla \bar{\mathcal{L}} = \nabla \mathcal{L} + \alpha \mathbf{w}$$

Non-essential weights shrink toward zero. Essential weights survive.

7.3 Data Augmentation

Create new training examples by applying label-preserving transformations: rotation, flipping, cropping, colour jitter, noise. Works well for vision; also shown to work for speech.

7.4 Parameter Sharing

Same weight applied to different inputs (e.g., CNN filters). Massively reduces effective parameter count.

7.5 Dropout

Definition

During each forward pass, **randomly disable** each neuron independently:

- Visible (input) units retained with probability $p = 0.8$.
- Hidden units retained with probability $p = 0.5$.

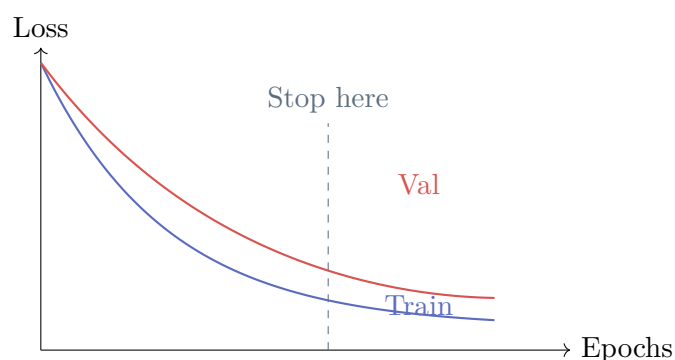
At test time, use all neurons but scale weights by p .

Intuition

Each mini-batch trains a different *thinned network*. With n hidden units there are 2^n possible networks. Dropout approximates averaging over all of them (model ensemble) cheaply. Prevents **co-adaptation**: neurons can't rely on specific other neurons being present.

7.6 Early Stopping

Monitor *validation* loss during training. Stop when validation loss stops decreasing (patience p steps). Return the model from p steps before stopping.



7.7 Ensemble Methods & Bagging

Train k models on different bootstrap samples (sampling with replacement). Average predictions.

Key condition: errors must be *uncorrelated* for ensembles to help.

Exam Tip

Dropout vs. Bagging: Bagging trains k independent models. Dropout approximates an exponentially large ensemble by weight-sharing across thinned networks — much cheaper.

Injecting noise at input $\approx$ L2 regularisation (can be proved for linear models).

Common Mistake

Dropout is applied only during **training**. At inference, disable dropout and scale weights by the keep probability p . Forgetting this is a common implementation bug.

Quick Revision

- $\text{MSE} = \text{Bias}^2 + \text{Variance} + \text{irreducible noise}$.
- L2 reg: adds $\alpha \mathbf{w}$ to gradient (weight decay).
- Dropout ($p = 0.5$ hidden, 0.8 input): ensemble approximation.
- Early stopping: patience-based, returns best val-loss checkpoint.

Activation Functions & Deep Revival

8.1 Problems with Sigmoid & Tanh

Function	Range	Derivative	Problems
Sigmoid σ	$(0, 1)$	$\sigma(1 - \sigma)$	Vanishing grad; not zero-centred;
Tanh	$(-1, 1)$	$1 - \tanh^2$	expensive exp Vanishing grad; expensive exp

Saturated neurons ($\sigma \approx 0$ or 1): gradient $\approx 0 \Rightarrow$ updates stop (**vanishing gradient**).

8.2 ReLU Family

Definition

$$\text{ReLU}(x) = \max(0, x), \quad g'(x) = \mathbf{1}[x > 0]$$

Advantages: no saturation for $x > 0$; computationally cheap; faster convergence in practice.

Dead ReLU: if pre-activation always ≤ 0 , neuron never updates. Fix: initialise bias > 0 .

Variant	Formula	Advantage
Leaky ReLU	$\max(0.1x, x)$	No dead neurons
PReLU	$\max(\alpha x, x)$, α learned	Adaptive
ELU	x if $x > 0$; $a(e^x - 1)$ if $x \leq 0$	Zero-centred; smooth at 0
MaxOut	$\max(h_1, h_2, \dots)$	Generalises ReLU & Leaky ReLU
GELU	$x \cdot \Phi(x)$ (Gaussian CDF)	Used in BERT, GPT
Swish	$x \cdot \sigma(\beta x)$	$\beta = 1.702 \approx \text{GELU}$

Exam Tip

ReLU derivative is exactly 0 or 1 — no vanishing gradient for positive pre-activations. This is why deep networks became trainable with ReLU.

GELU $= x \cdot \Phi(x)$ where Φ is Gaussian CDF. Modern transformers (BERT, GPT)

use GELU.

8.3 Unsupervised Pre-Training (Hinton, 2006)

Before fine-tuning with supervised labels, train greedily layer by layer to *reconstruct* the input:

1. Train h_1 to reconstruct $\mathbf{x}$.
2. Train h_2 to reconstruct h_1 .
3. ...
4. Add output layer, fine-tune all weights with supervised objective.

Benefits: good weight initialisation; prevents greedy overfitting to supervised signal; more robust to random initialisation.

8.4 Weight Initialisation

- **Xavier/Glorot:** $W \sim \mathcal{U}(-\sqrt{6/(n_{in} + n_{out})}, +\sqrt{6/(n_{in} + n_{out})})$. Good for tanh/sigmoid.
- **He/Kaiming:** $W \sim \mathcal{N}(0, 2/n_{in})$. Good for ReLU.

Quick Revision

- Sigmoid/tanh: vanishing gradient + non-zero-centred (sigmoid).
- ReLU: no saturation for $x > 0$; dead neuron for $x \leq 0$.
- Leaky ReLU: small slope for $x < 0$ fixes dead neurons.
- GELU $\approx$ Swish ($\beta = 1.702$) — modern default for transformers.
- Pre-training: greedy layer-wise reconstruction before supervised fine-tuning.

Convolutional Neural Networks

9.1 Motivation

Fully-connected layers on images are impractical: $227 \times 227 \times 3 = 154,587$ inputs $\rightarrow$ millions of parameters per layer. CNNs exploit **spatial locality** and **translation invariance**.

9.2 Convolution Operation

Definition

1D convolution: $(x * w)(t) = \sum_a x(a) w(t - a)$

2D convolution (for images): $(I * K)(i, j) = \sum_{a=0}^{m-1} \sum_{b=0}^{n-1} I(i - a, j - b) K(a, b)$

Cross-correlation (what libraries actually implement): $g(i, j) = \sum_a \sum_b I(i + a, j + b) K(a, b)$

Cross-correlation = convolution when K is symmetric. PyTorch/TensorFlow implement cross-correlation.

9.3 Output Dimensions

Input $W_1 \times H_1 \times D_1$, filter size F , K filters, stride S , padding P :

$$W_2 = \frac{W_1 - F + 2P}{S} + 1, \quad H_2 = \frac{H_1 - F + 2P}{S} + 1, \quad D_2 = K$$

Exam Tip

Parameter count for a conv layer with K filters of size $F \times F \times D_1$:

Weights: $K \times F \times F \times D_1$. Biases: K . Total: $K(F^2 D_1 + 1)$.

Worked example (PYQ-style): Input 3 channels, first conv layer 3×3 , 4 output channels.

Weights: $4 \times 3 \times 3 \times 3 = 108$. Biases: 4. Total: 112.

9.4 Pooling

Replaces a region with its max/average — reduces spatial dimensions, extracts dominant features.

- **Max pooling:** gradient flows only to the max-position element.
- **Average pooling:** gradient distributed equally across the region.
- No learnable parameters; reduces overfitting.

9.5 Backpropagation in CNNs

- **Through conv layer:** gradient update = convolve input X with the upstream gradient.
- **Propagating gradient:** convolve the filter W (flipped) with upstream gradient.
- **Weight sharing:** one filter applied at all positions $\Rightarrow$ gradients sum over all positions.

9.6 Famous Architectures

Network	Input / Key	Innovation
AlexNet (2012)	$227 \times 227 \times 3$; $\sim 27\text{M}$ params	ReLU, dropout, GPUs; ImageNet breakthrough
ZFNet	Same as AlexNet	Filters 512/1024/512 in middle layers
VGGNet	$224 \times 224 \times 3$	Only 3×3 kernels stacked deep
GoogLeNet	Inception module	Multiple filter sizes in parallel (1×1 , 3×3 , 5×5)
ResNet	Skip connections	Add identity shortcut $\mathbf{h}(x) = f(x) + x$; train very deep nets

Intuition

ResNet skip connections: the network only needs to learn the *residual* $f(\mathbf{x}) = \mathbf{h}(\mathbf{x}) - \mathbf{x}$. If a layer is useless, it can learn $f(\mathbf{x}) = 0$ and become an identity — depth doesn't hurt.

Common Mistake

1×1 convolutions are not trivial — they mix channels without spatial aggregation and are used in GoogLeNet to reduce dimensions cheaply.

Quick Revision

- $W_2 = (W_1 - F + 2P)/S + 1$ — know this formula.
- Params per conv layer: $K(F^2 D_1 + 1)$.
- Max pooling: gradient to max element only.
- ResNet: $\mathbf{h}(\mathbf{x}) = f(\mathbf{x}) + \mathbf{x}$ (skip connection).
- Libraries use cross-correlation, not true convolution.

Word Representations

10.1 The Problem with One-Hot Vectors

Vocabulary V : each word is a $|V|$ -dimensional binary vector. Two words that are semantically similar have:

- Same Euclidean distance: $\sqrt{2}$ for every pair.
- Cosine similarity: 0 for every pair.

Cat and dog are just as “similar” as cat and truck. We need **dense representations** that capture meaning.

10.2 Co-occurrence Matrix

Definition

Given context window k , X_{ij} = count of how often word j appears within k words of word i .

Problems: very large ($|V| \times |V|$), sparse, counts dominated by stop words.

Fix stop-word problem with **PMI**:

$$\text{PMI}(w, c) = \log \frac{P(c|w)}{P(c)} = \log \frac{\text{count}(w, c) \cdot N}{\text{count}(c) \cdot \text{count}(w)}$$

PPMI: set PMI to 0 when $\text{count}(w, c) = 0$ (avoids $-\infty$).

10.2.1 SVD for Dense Representations

$$X_{\text{PPMI}} \approx \hat{X} = U\Sigma V^T \quad (\text{rank-}k \text{ approx.})$$

Word representation: $W_{\text{word}} = U\Sigma \in \mathbb{R}^{|V| \times k}$.

10.3 Prediction-Based Models (Neural Word Embeddings)

10.3.1 Bag-of-Words (CBOW)

Predict the target word given context words. Hidden layer = sum of context word embeddings:

$$\mathbf{h} = \sum_{i=1}^d \mathbf{u}_{c_i}, \quad \hat{y}_w = \text{softmax}(\mathbf{v}_w^T \mathbf{h})$$

Parameters: $W_{\text{context}} \in \mathbb{R}^{k \times |V|}$, $W_{\text{word}} \in \mathbb{R}^{|V| \times k}$.

Update rule: $\mathbf{v}_w \leftarrow \mathbf{v}_w + \eta \mathbf{u}_c (1 - \hat{y}_w)$ (increases cosine similarity between context and word).

10.3.2 Skip-Gram

Predict context words given a target word (role of context and word swapped vs. CBOW).

Problem with both: softmax denominator sums over all $|V|$ words — very expensive.

10.3.3 Negative Sampling

Instead of softmax over all words, train a binary classifier: real (w, c) pairs vs. random “negative” pairs (w, r) :

$$\max \sum_{(w,c) \in D} \log \sigma(\mathbf{v}_c^T \mathbf{v}_w) + \sum_{(w,r) \in D'} \log \sigma(-\mathbf{v}_r^T \mathbf{v}_w)$$

10.3.4 Hierarchical Softmax

Build a binary tree over $|V|$ words. Probability = product along the unique path from root to word leaf. Cost: $O(\log |V|)$ instead of $O(|V|)$.

10.4 GloVe (Global Vectors)

Combine count-based (SVD) and prediction-based approaches:

$$\mathbf{v}_i^T \mathbf{v}_j + b_i + b_j = \log X_{ij}$$

$$\min \sum_{i,j} f(X_{ij}) (\mathbf{v}_i^T \mathbf{v}_j + b_i + b_j - \log X_{ij})^2$$

where $f(x) = (x/x_{\max})^\alpha$ for $x < x_{\max}$, else 1 (downweights very frequent pairs).

Exam Tip

Count-based (SVD, GloVe): use global co-occurrence matrix; fast to train.

Prediction-based (Word2Vec): local context; negative sampling makes it scalable.

GloVe: best of both worlds — global counts + prediction objective.

Quick Revision

- One-hot: cosine sim = 0 for all pairs; no semantic info.
- PPMI: removes stop-word dominance; clip PMI at 0.
- SVD: $X \approx U\Sigma V^T$; $W_{\text{word}} = U\Sigma$.
- CBOW: context $\rightarrow$ word; Skip-gram: word $\rightarrow$ context.
- Negative sampling: binary classifier, avoid full softmax.
- GloVe: $\mathbf{v}_i^T \mathbf{v}_j + b_i + b_j = \log X_{ij}$.

Recurrent Neural Networks

11.1 Why RNNs?

Standard FFNNs assume fixed-size, independent inputs. Many real tasks require:

- **Variable-length** sequences (sentences, audio, video).
- **Dependent** inputs (word t depends on words $1 \dots t - 1$).

11.2 RNN Architecture

Definition

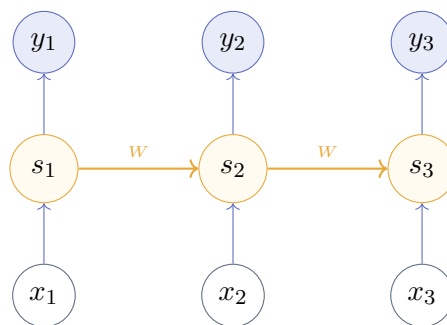
At each time step t , the RNN computes:

$$\mathbf{s}_t = \sigma(U\mathbf{x}_t + W\mathbf{s}_{t-1} + \mathbf{b}) \quad \mathbf{y}_t = O(V\mathbf{s}_t + \mathbf{c})$$

Parameters U, W, V are **shared across all time steps** (parameter sharing).

Dimensions: $\mathbf{x}_t \in \mathbb{R}^n$, $\mathbf{s}_t \in \mathbb{R}^d$, $\mathbf{y}_t \in \mathbb{R}^k$:

$$U \in \mathbb{R}^{d \times n}, \quad W \in \mathbb{R}^{d \times d}, \quad V \in \mathbb{R}^{k \times d}$$



11.3 Backpropagation Through Time (BPTT)

Total loss: $\mathcal{L} = \sum_t \mathcal{L}_t$.

Gradient of W involves a chain of products:

$$\frac{\partial \mathcal{L}_t}{\partial W} = \sum_{k=1}^t \frac{\partial \mathcal{L}_t}{\partial \mathbf{s}_t} \left(\prod_{j=k+1}^t \frac{\partial \mathbf{s}_j}{\partial \mathbf{s}_{j-1}} \right) \frac{\partial \mathbf{s}_k}{\partial W}$$

Each factor $\frac{\partial s_j}{\partial s_{j-1}} = W^T \text{diag}(\sigma'(s_{j-1}))$.

Vanishing gradient: if $\|W\| < 1$ or $\sigma' \ll 1$, the product $\rightarrow 0$ exponentially.

Exploding gradient: if $\|W\| > 1$, product $\rightarrow \infty$. Fix: **gradient clipping**.

Truncated BPTT: limit backprop to τ steps.

11.4 RNN Modes

Mode	Description
One-to-many	One image $\rightarrow$ sequence of words (image captioning)
Many-to-one	Sequence of words $\rightarrow$ one sentiment label
Many-to-many (sync)	POS tagging: $\text{word}_t \rightarrow \text{tag}_t$
Many-to-many (async)	Encoder: read full sequence; Decoder: generate output

Exam Tip

Key insight on parameter sharing: same U, W, V for all t means:

1. The network is input-size agnostic.
2. Long sequences don't add new parameters.
3. But BPTT through long sequences $\Rightarrow$ vanishing/exploding gradients.

Quick Revision

- RNN: $\mathbf{s}_t = \sigma(U\mathbf{x}_t + W\mathbf{s}_{t-1} + \mathbf{b})$.
- Parameters: U (input), W (recurrent), V (output).
- BPTT: chain of t Jacobian multiplications $\rightarrow$ vanishing/exploding.
- Gradient clipping for exploding; LSTM for vanishing.
- Truncated BPTT: only backprop through τ steps.

LSTM & GRU

12.1 The Problem LSTM Solves

By the time an RNN reads a long document, information from the first few words is largely lost — the hidden state gets overwritten at every step.

Ideal behaviour: *forget* stop words, *selectively read* sentiment words, *selectively write* new information.

12.2 Long Short-Term Memory (LSTM)

Definition

LSTM maintains two state vectors: **cell state** $\mathbf{s}_t$ (long-term) and **hidden state** $\mathbf{h}_t$ (short-term).

Three **gates** (each $\in (0, 1)^d$):

$$\text{Output gate: } \mathbf{o}_t = \sigma(W_o \mathbf{h}_{t-1} + U_o \mathbf{x}_t + \mathbf{b}_o)$$

$$\text{Input gate: } \mathbf{i}_t = \sigma(W_i \mathbf{h}_{t-1} + U_i \mathbf{x}_t + \mathbf{b}_i)$$

$$\text{Forget gate: } \mathbf{f}_t = \sigma(W_f \mathbf{h}_{t-1} + U_f \mathbf{x}_t + \mathbf{b}_f)$$

Candidate cell state and update:

$$\tilde{\mathbf{s}}_t = \sigma(W \mathbf{h}_{t-1} + U \mathbf{x}_t + \mathbf{b})$$

$$\mathbf{s}_t = \mathbf{f}_t \odot \mathbf{s}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{s}}_t$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \sigma(\mathbf{s}_t)$$

Intuition

- $\mathbf{f}_t \approx 0$: **forget** old cell state.
- $\mathbf{i}_t \approx 0$: **ignore** new candidate.
- $\mathbf{o}_t \approx 0$: **hide** cell from output.
- Gates are soft (0 to 1), learned jointly with other parameters.

Why does LSTM fix vanishing gradient? The cell state update $\mathbf{s}_t = \mathbf{f}_t \odot \mathbf{s}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{s}}_t$ has an additive path — gradient can flow back without multiplicative decay as long as $\mathbf{f}_t \approx 1$.

12.3 Gated Recurrent Unit (GRU)

Simpler variant with 2 gates (no separate cell state):

$$\begin{aligned}\mathbf{o}_t &= \sigma(W_o \mathbf{s}_{t-1} + U_o \mathbf{x}_t + \mathbf{b}_o) && \text{(output/reset gate)} \\ \mathbf{i}_t &= \sigma(W_i \mathbf{s}_{t-1} + U_i \mathbf{x}_t + \mathbf{b}_i) && \text{(input/update gate)} \\ \tilde{\mathbf{s}}_t &= \sigma(W(\mathbf{o}_t \odot \mathbf{s}_{t-1}) + U \mathbf{x}_t + \mathbf{b}) \\ \mathbf{s}_t &= (1 - \mathbf{i}_t) \odot \mathbf{s}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{s}}_t\end{aligned}$$

Key difference: forget gate and input gate are *tied* ($\mathbf{f}_t = 1 - \mathbf{i}_t$). Fewer parameters than LSTM.

12.4 Vanishing Gradient in LSTM — Forward vs. Backward

During **forward propagation**: gates prevent irrelevant info from entering the cell state.

During **backward propagation**: gates control gradient flow. If $\mathbf{s}_{t-1}$ didn't contribute to $\mathbf{s}_t$ (low $\mathbf{f}_t$), gradient into $\mathbf{s}_{t-1}$ is small — but that's *correct* behaviour. True long-range dependencies have $\mathbf{f}_t \approx 1$, allowing gradient to flow freely.

	RNN	LSTM	GRU
Gates	None	3 ($\mathbf{o}, \mathbf{i}, \mathbf{f}$)	2 ($\mathbf{o}, \mathbf{i}$)
States	1 ($\mathbf{s}$)	2 ($\mathbf{s}, \mathbf{h}$)	1 ($\mathbf{s}$)
Vanishing gradient	Severe	Largely solved	Largely solved
Params	Fewest	Most	Intermediate

Memory Trick

LSTM gates mnemonic — FOI:

Forget old cell · Output to hidden · Input new candidate.

Cell update: $\mathbf{s}_t = \mathbf{f} \odot \mathbf{s}_{t-1} + \mathbf{i} \odot \tilde{\mathbf{s}}$.

Hidden: $\mathbf{h}_t = \mathbf{o} \odot \tanh(\mathbf{s}_t)$.

Quick Revision

- LSTM: cell state + hidden state; forget, input, output gates.
- Cell update: $\mathbf{s}_t = \mathbf{f}_t \odot \mathbf{s}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{s}}_t$.
- Hidden: $\mathbf{h}_t = \mathbf{o}_t \odot \sigma(\mathbf{s}_t)$.
- GRU: 2 gates; forget = 1 − input (tied).
- Additive cell update = highway for gradients.

Encoder-Decoder Models & Attention

13.1 Encoder-Decoder Architecture

Used when input and output are both sequences of *different* lengths.

Definition

Encoder: reads the full input sequence, produces a fixed-size context vector $\mathbf{c}$.

Decoder: generates the output sequence conditioned on $\mathbf{c}$.

Two ways to condition decoder on image/context I :

1. Set $\mathbf{s}_0 = \text{fc7}(I)$ (CNN representation): all subsequent states depend on I .
2. Explicit: $\mathbf{s}_t = \text{RNN}(\mathbf{s}_{t-1}, [\mathbf{x}_t; \text{fc7}(I)])$ at every step.

Applications: machine translation, image captioning, video captioning, textual entailment, VQA.

13.2 Bottleneck Problem

Compressing a long sequence into a single fixed vector $\mathbf{c}$ loses information. The decoder must reconstruct the full source from this bottleneck.

13.3 Attention Mechanism

Definition

At each decode step t , compute an importance score for each encoder hidden state $\mathbf{h}_j$:

$$e_{jt} = f_{\text{ATT}}(\mathbf{s}_{t-1}, \mathbf{h}_j), \quad \alpha_{jt} = \frac{\exp(e_{jt})}{\sum_{j'} \exp(e_{j't})}$$

Then take a weighted sum as the context vector for step t :

$$\mathbf{c}_t = \sum_j \alpha_{jt} \mathbf{h}_j$$

α_{jt} = probability of attending to source position j when decoding position t .

One choice for f_{ATT} (Bahdanau / additive):

$$e_{jt} = \mathbf{V}_{\text{att}}^T \tanh(U_{\text{att}} \mathbf{s}_{t-1} + W_{\text{att}} \mathbf{h}_j)$$

13.4 Attention for Images

Instead of fc7 (global feature), use a convolutional feature map (e.g., VGGNet conv5: $14 \times 14 \times 512 = 196$ spatial locations). Attention is now over spatial locations.

13.5 Hierarchical Attention

Documents = sequences of sentences; sentences = sequences of words.

1. **Word-level RNN**: encode each sentence $\rightarrow$ sentence vector.
2. **Word-level attention**: weight words within each sentence.
3. **Sentence-level RNN**: encode sequence of sentence vectors $\rightarrow$ document vector.
4. **Sentence-level attention**: weight sentences within the document.

Exam Tip

Attention weights can be visualised as a **heatmap**: rows = output positions, columns = input positions. A well-trained model shows near-diagonal structure for aligned languages.

Attention does NOT fix: sequential computation (RNN still processes $\mathbf{s}_1 \rightarrow \mathbf{s}_2 \rightarrow \dots$). This is what Transformers fix.

Quick Revision

- Encoder $\rightarrow$ fixed vector $\mathbf{c}$ (bottleneck).
- Attention: learned per-step context $\mathbf{c}_t = \sum_j \alpha_{jt} \mathbf{h}_j$.
- $\alpha_{jt} = \text{softmax}(e_{jt})$, $e_{jt} = f_{\text{ATT}}(\mathbf{s}_{t-1}, \mathbf{h}_j)$.
- Hierarchical: word-level then sentence-level attention.
- RNN bottleneck remains; Transformer eliminates it.

Transformer Architecture

14.1 Motivation

RNNs process tokens **sequentially**: $\mathbf{s}_t$ depends on $\mathbf{s}_{t-1} \Rightarrow$ cannot parallelise across time steps.

Transformers compute all positions **in parallel** using self-attention.

14.2 Self-Attention

For each word, compute three vectors by projecting its embedding $\mathbf{h}_j$:

$$\mathbf{q}_j = W_Q \mathbf{h}_j \quad (\text{query}), \quad \mathbf{k}_j = W_K \mathbf{h}_j \quad (\text{key}), \quad \mathbf{v}_j = W_V \mathbf{h}_j \quad (\text{value})$$

Attention score between positions i and j : $e_{ij} = \mathbf{q}_i^T \mathbf{k}_j / \sqrt{d_k}$

Output for position i : $\mathbf{z}_i = \sum_j \alpha_{ij} \mathbf{v}_j$, where $\alpha_{ij} = \text{softmax}(e_{ij})$

Matrix form:

$$Z = \text{softmax}\left(\frac{Q^T K}{\sqrt{d_k}}\right) V^T \quad Q = W_Q H, \quad K = W_K H, \quad V = W_V H$$

Intuition

Query = “what I’m looking for”. Key = “what I offer”. Value = “what I actually give”.

$\mathbf{q}_i^T \mathbf{k}_j$: how relevant is word j to word i ? Scale by $1/\sqrt{d_k}$ to prevent dot products from growing too large (which would push softmax to very peaked distributions).

14.3 Multi-Headed Attention

Run h self-attention heads *in parallel*, concatenate outputs:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(h)}) W_O$$

Analogous to **multiple filters in CNNs** — each head attends to different aspects of the input.

14.4 Positional Encoding

Self-attention is permutation-invariant (no notion of order). Add positional encodings to embeddings:

$$\text{PE}_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad \text{PE}_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

14.5 Feed-Forward Sub-layer

After attention, each position independently passes through a 2-layer MLP:

$$\text{FFN}(\mathbf{z}) = \max(0, W_1 \mathbf{z} + \mathbf{b}_1) W_2 + \mathbf{b}_2$$

Same MLP for all positions (parameter sharing across sequence), different parameters per layer.

14.6 Encoder Architecture

Each encoder layer = **Multi-head self-attention** + **Add & Norm** + **FFN** + **Add & Norm**. Stack $N = 6$ identical encoder layers.

Parallelisation: within a layer, all positions are computed simultaneously.

14.7 Decoder Architecture

Each decoder layer = **Masked self-attention** + **Cross-attention** + **FFN**.

- **Masked self-attention:** when generating token t , mask all future positions ($> t$) by setting scores to $-\infty$ before softmax.
- **Cross-attention:** queries from decoder, keys and values from encoder output.

Component	Purpose	Complexity
Self-attention	Capture all-pair dependencies	$O(T^2d)$ per layer
Multi-head	Multiple relation types in parallel	h heads, d/h each
Positional encoding	Inject sequence order	$O(1)$ (no params)
FFN	Per-position transformation	$O(Td^2)$ per layer
Masking (decoder)	Prevent attending to future	$-\infty$ before softmax

Exam Tip

RNN vs. Transformer (PYQ Statement I & II):

Statement I: Transformers allow parallel computation across tokens during training; RNNs process sequentially. **TRUE**.

Statement II: Transformer reduces training time significantly for long sequences vs. RNN. **TRUE**.

Statement I IS the correct explanation of Statement II.

Memory Trick

Transformer = Attention is all you need:

Q(uey), K(ey), V(alue) — like a lookup table.

Scale by $\sqrt{d_k}$ to avoid peaked softmax.

Multi-head = multiple CNNs filters.

Positional encoding = stamp the order in.

Quick Revision

- Self-attn: $Z = \text{softmax}(Q^T K / \sqrt{d_k}) V^T$ — all in parallel.
- Scale $1/\sqrt{d_k}$: prevents softmax saturation.
- Multi-head: concatenate h heads $\rightarrow$ project.
- Positional encoding: sinusoidal, added to embeddings.
- Decoder: masked self-attn (causal) + cross-attn to encoder.
- Masking: $-\infty$ at future positions.

Quick Reference Cheat Sheet

Key Equations

Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$, $\sigma' = \sigma(1 - \sigma)$

Softmax: $\hat{y}_j = \frac{e^{a_j}}{\sum_k e^{a_k}}$

Cross-entropy output gradient:
 $\nabla_{\mathbf{a}_L} = \hat{\mathbf{y}} - \mathbf{e}_y$

Conv output size: $\frac{W-F+2P}{S} + 1$

Conv params: $K(F^2 D_{in} + 1)$

Adam: $\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t}$, $\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t}$

LSTM cell: $\mathbf{s}_t = \mathbf{f}_t \odot \mathbf{s}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{s}}_t$

LSTM hidden: $\mathbf{h}_t = \mathbf{o}_t \odot \sigma(\mathbf{s}_t)$

Self-attention: $Z = \text{softmax}\left(\frac{Q^T K}{\sqrt{d_k}}\right) V^T$

GloVe: $\mathbf{v}_i^T \mathbf{v}_j + b_i + b_j = \log X_{ij}$

Architecture Timeline

AlexNet (2012) $\rightarrow$ VGG $\rightarrow$ GoogLeNet (Inception) $\rightarrow$ ResNet (2015) $\rightarrow$ LSTM (1997) $\rightarrow$ Attention (2015) $\rightarrow$ Transformer (2017) $\rightarrow$ BERT/GPT

Regularisation Methods

L2 (weight decay), Dropout (0.5 hidden, 0.8 input), Early stopping, Data augmentation, Parameter sharing, Bagging.

Activation Functions

Function	Range	Issue
Sigmoid	(0,1)	Vanishing, not 0-centered
Tanh	(-1,1)	Vanishing
ReLU	$[0, \infty)$	Dead neurons
Leaky ReLU	$\mathbb{R}$	None
GELU	$\mathbb{R}$	None (best)

Gradient Descent Family

Batch $\rightarrow$ SGD $\rightarrow$ Mini-batch $\rightarrow$ Momentum $\rightarrow$ NAG $\rightarrow$ AdaGrad $\rightarrow$ RMSprop $\rightarrow$ AdaDelta $\rightarrow$ Adam $\rightarrow$ NAdam

PYQ Solutions

15.1 End Term Jan 2026 — With Solutions

PYQ: Q2 — Convolution Output [2 marks]

Question: Input matrix I convolved with filter K , padding $P = 1$, stride $S = 2$. Find value at specific output position.

Answer: 3

Solution method:

1. Pad input with zeros ($P=1$): add border of 0s around I .
2. Slide filter K with stride 2 across padded input.
3. Output size: $\frac{W+2P-F}{S} + 1$.
4. Compute element-wise product + sum at the queried position.

PYQ: Q3 — Gradient Descent Steps [3 marks]

Question: Minimise $f(x)$ starting at x_0 , two steps of GD with learning rate η .

Answer: $x_2 \in [0.14, 0.18]$

Solution template:

$$x_1 = x_0 - \eta f'(x_0), \quad x_2 = x_1 - \eta f'(x_1)$$

Substitute the given x_0 and η , compute f' at each step.

PYQ: Q4 — AdaGrad Check [3 marks]

Question: Starting at x_0 , AdaGrad update. Does a constant c exist such that $x_t = c$ for all t ? If so find c , else enter -1 .

Answer: 0 (i.e., $c = 0$; the stationary point is at $x = 0$)

Reasoning: AdaGrad: $v_t = v_{t-1} + (f'(x_{t-1}))^2$, $x_t = x_{t-1} - \frac{\eta}{\sqrt{v_t + \epsilon}} f'(x_{t-1})$. For $x_t = c$ (constant), we need $f'(c) = 0$. Minimum of the given function is at $x = 0$.

PYQ: Q5 — Forward Pass & Loss [3 marks]

Question: Network: 1 hidden layer (3 ReLU neurons), 1 output (sigmoid). Squared loss $\mathcal{L} = \frac{1}{2}(y - \hat{y})^2$. Input x , true y . Compute loss.

Answer: 0.125

Solution template:

1. Compute hidden pre-activations: $a_i = w_i \cdot x$ (bias = 0).
2. Apply ReLU: $h_i = \max(0, a_i)$.
3. Output pre-activation: $a_{out} = \sum_i w_{2,i} h_i$.
4. Apply sigmoid: $\hat{y} = \sigma(a_{out})$.
5. Loss: $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2 = 0.125$.

PYQ: Q6 — Weights Updated in Backprop [2 marks]

Question: After forward pass, perform backprop with SGD. How many weights in W_1 (first layer) get updated?

Answer: 0

Reasoning: If *all* hidden neurons have pre-activation ≤ 0 (ReLU dead), then $\nabla_{\mathbf{a}_k} = 0$ for the hidden layer. The gradient $\nabla_{W_1} = \nabla_{\mathbf{a}_1} \mathbf{x}^T = \mathbf{0}$, so no weights update.

PYQ: Q7 — Sigmoid vs. ReLU Backprop [2 marks]

Question: Would changing hidden activation to sigmoid change the answer to Q6?

Answer: Yes

Reasoning: Sigmoid $\sigma(a) \in (0, 1)$ for all a — it never fully saturates to exactly 0 (unlike dead ReLU). Therefore $\sigma'(a) > 0$ everywhere, gradients are non-zero, and weights *do* get updated.

PYQ: Q8 — CNN Parameter Count [3 marks]

Question: 2D CNN. Input: 3 channels. Conv layer 1: 3×3 kernel, 4 output channels. Conv layer 2: 5×5 kernel, 10 output channels. Which statements are correct?

Answer: Layer 1 has 108 weights. Layer 2 has 1000 weights. Layer 1 has 4 biases. Layer 2 has 10 biases.

Solution:

- Layer 1 weights: $4 \times 3 \times 3 \times 3 = 108$. (Correct)
- Layer 2 weights: $10 \times 5 \times 5 \times 4 = 1000$. (Correct)
- Layer 1 biases: 4 (one per output channel). (Correct)
- Layer 2 biases: 10. (Correct)

- “36 weights” for layer 1: $\frac{108}{3} = 36$? No — you *cannot* ignore input depth.
- “250 weights” for layer 2: wrong — $10 \times 5 \times 5 = 250$ ignores input depth 4.
- “12 biases” for layer 2: Wrong — biases = number of output channels = 10.

PYQ: Q9 — Regularisation Methods [2 marks]

Question: Which prevent overfitting?

Answer: Dropout, Early stopping, L2 regularisation (*not* MaxOut, *not* increasing depth)

Reasoning:

- Dropout: ensemble approximation, prevents co-adaptation.
- Early stopping: stops before overfitting.
- L2 regularisation: penalises large weights, reduces complexity.
- MaxOut: a powerful activation function, but not a regularisation technique per se.
- Increasing model depth: increases capacity $\Rightarrow$ more likely to overfit.

PYQ: Q10 — XOR with Neural Network [3 marks]

Question: XOR classification with 1 hidden layer (2 ReLU neurons), 1 output (linear). Second layer weights scaled by λ . Find range of λ for correct classification with threshold 0.5.

Key insight: The network must produce > 0.5 for XOR=1 inputs and < 0.5 for XOR=0 inputs. Working through the given weight matrices:

1. Compute hidden activations for all 4 inputs: (0, 0), (1, 0), (0, 1), (1, 1).
2. Apply ReLU.
3. Compute scaled output $= W_2^{(\lambda)} \mathbf{h}$.
4. Threshold at 0.5.

The valid range gives the correct answer among the four options.

PYQ: Q11 — Transformers vs. RNN [3 marks]

Question: Statement I: Transformers allow parallel computation across tokens during training; RNNs process tokens sequentially. Statement II: Transformer reduces training time significantly for long sequences compared to RNN.

Answer: Both I and II are true, and Statement I is the correct explanation of II.

Reasoning: RNNs compute $\mathbf{s}_t$ from $\mathbf{s}_{t-1}$ — strictly sequential. Transformers compute all positions in one matrix multiply $Z = \text{softmax}(Q^T K / \sqrt{d_k}) V^T$ in parallel. This *directly* causes the training speedup (Statement I $\Rightarrow$ Statement II).

15.2 Term 3 2025 — Key Questions

PYQ: T3-Q2 — Co-occurrence Matrix [3 marks]

Question: What is a co-occurrence matrix in NLP?

Answer: A matrix where rows and columns represent words, and each cell indicates how frequently two words co-occur within a context window.

Wrong options explained: “similarity scores between documents” is a document-document matrix (different thing). Co-occurrence is word $\times$ word (or word $\times$ context).

PYQ: T3-Q3 — Word Embeddings vs. One-Hot [2 marks]

Question: Primary advantage of word embeddings over one-hot?

Answer: Captures semantic relationships.

One-hot: all inter-word distances are equal ($\sqrt{2}$); cosine similarity 0 everywhere. Embeddings: semantically similar words have higher cosine similarity.

PYQ: T3-Q4 — OR with MP Neuron (MSQ) [3 marks]

Question: Implement OR with 2 binary inputs using McCulloch-Pitts neuron (no inhibitory input). Which $(\theta, \text{num_correctly_classified})$ pairs are valid?

Answer: $\theta = 1$ works. Truth table:

x_1	x_2	OR
0	0	0
1	0	1
0	1	1
1	1	1

$g(x_1, x_2) = x_1 + x_2$. $\theta = 1$: output 1 iff $g \geq 1$. Row 1: $g = 0 < 1 \Rightarrow y = 0$; rows 2-4: $g \geq 1 \Rightarrow y = 1$. All 4 correct.

15.3 Quiz Papers — Key Questions

PYQ: Quiz 1 Jan 2026 — Bias-Variance

Question: Model trained on complex architecture shows low training loss but high validation loss. Identify the statements about bias and variance.

Answer: Low bias, high variance (classic overfitting signature).

Low training loss $\Rightarrow$ model fits training data well $\Rightarrow$ low bias.

High validation loss $\Rightarrow$ model does not generalise $\Rightarrow$ high variance.

PYQ: Quiz 2 Jan 2026 — Optimiser Matching

Question: Match optimisers in Column A with their properties in Column B.

Answer template:

Optimiser	Key property
SGD	No adaptive LR; noisy updates
Momentum	Exponential moving average of gradients
AdaGrad	Accumulates squared gradients; LR decays to 0
RMSprop	Decaying average of squared gradients
Adam	Bias-corrected first + second moment estimates

PYQ: Quiz 2 Jan 2026 — Cyclical LR / Saddle Point [4 marks]

Question: Choose the best learning rate schedule to avoid saddle points.

Answer: **Cyclical learning rate** (triangular or cosine annealing with warm restarts).

Reasoning: A monotonically decreasing LR gets stuck at saddle points because the LR is too small to escape once reached. Cyclical LR periodically increases the LR, providing enough “kick” to escape.

Index of Key Formulas

- Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$
- $\sigma' = \sigma(1 - \sigma)$
- Tanh: $\tanh' = 1 - \tanh^2$
- ReLU: $\max(0, x)$, $\text{ReLU}' = \mathbf{1}[x > 0]$
- GELU: $x\Phi(x)$
- CE loss: $-\sum y_j \log \hat{y}_j$
- MSE: $\frac{1}{N} \sum \|\hat{\mathbf{y}} - \mathbf{y}\|^2$
- Softmax gradient: $\hat{\mathbf{y}} - \mathbf{e}_y$
- Momentum: $\mathbf{u}_t = \beta \mathbf{u}_{t-1} + \nabla \theta$
- NAG: $\mathbf{u}_t = \beta \mathbf{u}_{t-1} + \nabla(\theta - \beta \mathbf{u}_{t-1})$
- AdaGrad: $v_t = v_{t-1} + g_t^2$
- RMSprop: $v_t = \beta v_{t-1} + (1 - \beta)g_t^2$
- Adam $\hat{m}$: $m_t/(1 - \beta_1^t)$
- Adam $\hat{v}$: $v_t/(1 - \beta_2^t)$
- L2 grad: $\nabla \mathcal{L} + \alpha \mathbf{w}$
- Conv size: $(W - F + 2P)/S + 1$
- Conv params: $K(F^2 D + 1)$
- LSTM cell: $\mathbf{f}_t \odot \mathbf{s}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{s}}_t$
- LSTM hidden: $\mathbf{o}_t \odot \sigma(\mathbf{s}_t)$
- Self-attn: $Z = \text{softmax}(\frac{Q^T K}{\sqrt{d_k}})V^T$
- PMI: $\log \frac{p(c|w)}{p(c)}$
- GloVe: $\mathbf{v}_i^T \mathbf{v}_j + b_i + b_j = \log X_{ij}$
- Bias-var: $\text{MSE} = \text{Bias}^2 + \text{Var} + \sigma^2$
- Cosine LR: $\eta_{\min} + \frac{\eta_{\max} - \eta_{\min}}{2} (1 + \cos \pi \frac{t}{T})$